# COMP0245 Machine Learning for Robotics

Amir Gerges
*Robotics and Artificial Intelligence*
amir.gerges.25@ucl.ac.uk

## Lab 1 – Linear Regression of Robot Dynamics

### I. Methodology

The aim of this work is to practice using linear regression to estimate robot dynamic parameters (mass, centre of mass, inertia) and to predict torques for each joint on the Franka Emika Panda robot. This lab consists of four main procedures: simulation setup, data collection, linear regression, and results evaluation.

#### A. Simulation Setup

Since the environment is already configured by RoboEnv, we can directly use the dynamic script `dynamic_regression.py` to run simulations and control the robot. The dynamic parameters $a$ for each link can be found in `panda.urdf`. For the Franka Emika Panda robot, we focus on links 1 through 7, as link 0 corresponds to the stationary base and link 8 is fixed, and thus both are excluded from the analysis.

#### B. Linear Regression

To understand the foundation of this lab and the motivation behind data collection, we first introduce the concept of linear regression in robot dynamics.

*1) Robot Dynamics:* The dynamics of a robot manipulator can be expressed as:

$$M(q)\ddot{q} + C(q,\dot{q}) + g(q) = Y(q,\dot{q},\ddot{q})\,\boldsymbol{a} = \mathbf{u} \tag{1}$$

For a robot with $n$ active joints, the regressor matrix $Y(q,\dot{q},\ddot{q})$ has dimensions $n \times 10n$. The parameter vector $\boldsymbol{a} \in \mathbb{R}^{10n\times 1}$ is composed of $n$ groups of 10 parameters, each corresponding to the dynamics of a joint. The torque vector $\mathbf{u} \in \mathbb{R}^{n\times 1}$ represents the torques applied to each of the $n$ joints.

*2) Linear Regression:* A multiple linear regression model can be expressed in matrix form as:

$$\mathbf{y} = X\boldsymbol{\beta} + \boldsymbol{\varepsilon}, \tag{2}$$

where $\mathbf{y}$ is the vector of observations, $X$ is the design matrix, $\boldsymbol{\beta}$ contains the regression coefficients, and $\boldsymbol{\varepsilon}$ represents the residual errors.

The regression coefficients can be estimated using the Moore–Penrose pseudoinverse:

$$\boldsymbol{\beta} = X^{+}\mathbf{y} = (X^{\top}X)^{-1}X^{\top}\mathbf{y}. \tag{3}$$

The pseudoinverse $X^{+}$ can be calculated using the `numpy.linalg.pinv` function, which employs singular value decomposition (SVD) by default, ensuring high numerical stability.

Applying this framework to robot dynamics, the dynamic parameters $\boldsymbol{a}$ can be estimated as:

$$\hat{\boldsymbol{a}} = Y^{+}\mathbf{u}. \tag{4}$$

And, if we consider contributions from all links to final torques and divide them into the contribution given by links 1 to 6 and by link 7, we can get:

$$u = Y_{1\text{–}6}a_{1\text{–}6} + Y_7 a_7, \tag{5}$$

where $u \in \mathbb{R}^{N\times 1}$, $Y_{1\text{–}6} \in \mathbb{R}^{N\times 60}$, and $Y_7 \in \mathbb{R}^{N\times 10}$.

According to (5), for the subset corresponding to the seventh link, as all the physical parameters from links 1 to 6 are known, we can know their contributions to final torques and compute $\hat{\boldsymbol{a}}_7$ using:

$$\hat{a}_7 = Y_7^{+}(u - Y_{1\text{–}6}a_{1\text{–}6}). \tag{6}$$

The predicted torques can then be calculated as:

$$\hat{\mathbf{u}} = Y\hat{\boldsymbol{a}}, \quad \hat{\mathbf{u}}_7 = Y_7\hat{\boldsymbol{a}}_7. \tag{7}$$

*3) Intercept Term Explanation:* In general linear regression, it is common to include an intercept term in the feature matrix to account for bias. However, for the robot dynamics described in (1), an intercept term is not required. Physically, this can be understood as follows: if the robot is stationary (i.e., $q = \dot{q} = \ddot{q} = 0$), then

- The dynamic regressor matrix $Y$ becomes a zero matrix.
- The required joint torques $\boldsymbol{\tau}$ are also zero.

$$Y = 0 \quad \Rightarrow \quad \boldsymbol{\tau} = 0$$

Consequently, the regression model naturally passes through the origin, and no intercept term is needed.

#### C. Data Collection

Data are collected throughout the simulation and processed afterward.

*1) Data Acquisition:* As introduced in (1), at each time step we use the functions `GetMotorAngles`, `GetMotorVelocities`, `ComputeMotorAccelerationTMinusOne`, and `GetMotorTorques` to obtain $q(t)$, $\dot{q}(t)$, $\ddot{q}(t)$, and $\mathbf{u}(t)$, respectively. The dynamic regressor $Y_t$ was obtained using Pinocchio's built-in `ComputeDynamicRegressor` function through the `PinWrapper` interface.

*2) Data Export:* To facilitate post-analysis and avoid mixing data collection with simulation, the variables are saved using `numpy.savez`:

$$Y_7, \quad Y, \quad \hat{\boldsymbol{a}}_7, \quad \hat{\boldsymbol{a}}, \quad \boldsymbol{a}, \quad \mathbf{u}_7, \quad \mathbf{u}, \quad \hat{\mathbf{u}}_7, \quad \hat{\mathbf{u}}$$

### D. Results Evaluation

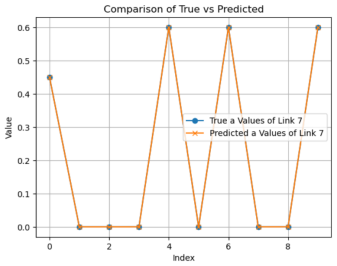
To assess the performance of the linear regression model, we computed the $F$-statistic, $R^2$ (coefficient of determination), adjusted $R^2$, and confidence intervals.

Note that, since the dynamics regression has no intercept term, we use $n-p$ as the degrees of freedom and apply slight modifications to the standard formulas.

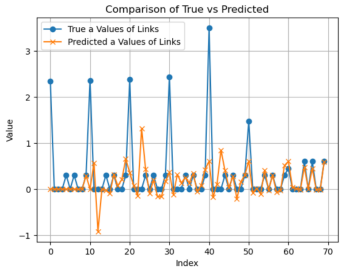
## II. Results

### A. Estimated Results of $a$ Parameters

Using (6) and (4), we obtained the predicted $a$ parameters and compared them with the true values.




(a) Link 7




(b) All Links

Fig. 1. Comparison of predicted and true $a$ parameters.

As shown in Fig. 1, with the information of links 1 to 6, we can get totally congruent $a$ parameters for link 7. While when we do the regression for all links, some parameters can not be precisely estimated because part 2 loses information about links 1 to 6 in part 1.

Initially, in part 1, when we estimated the dynamic parameter $a_7$ for link 7, the result did not fully align with the true value. Upon observing that the prediction exhibited inconsistent precision during the initial seconds, we decided to exclude the early data segment with giving up motion data in the first second. Surprisingly, this adjustment yielded an $a_7$ value that was entirely consistent with the ground truth. However, when we used this estimated dynamic parameters to predict torques of link 7, the $\hat{\boldsymbol{a}}_7$ with or without data from the first second did not show great differences in prediction performances. Similarly, we compared the regression results for all links in Part 2 using datasets with and without the first second of data. Interestingly, the model that included the initial data yielded slightly better F-statistics and adjusted $R^2$ values than the one that excluded it. We research that this phenomenon may be ascribable to two reasons:

- The initial phase of the robot's motion is transient, so using data from more stable periods can better capture the underlying relationship between $Y$ and $u$.
- Although the early data come from an unstable phase, excluding them may lead to information loss and increased prediction error at the beginning of the motion, especially when torques need to be predicted over the entire duration.

### B. Predicted Torque Results

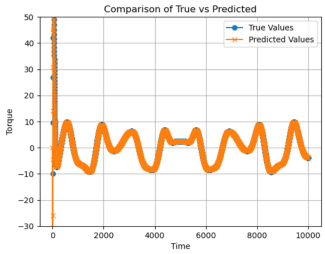



Fig. 2. Torque predictions for link 7.

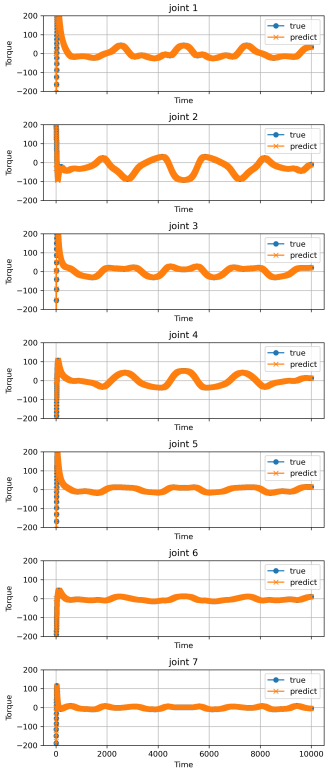


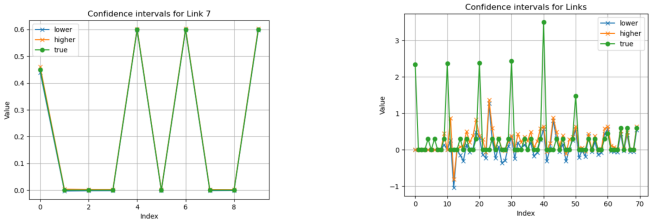

Fig. 3. Torque predictions for all links.

Despite some errors in the parameter estimates, the linear regression model demonstrates strong performance in predicting joint torques, with errors mainly happening in the beginning of the motion.

### C. F-statistics and Adjusted $R^2$

The F-statistic for link 7 is 29920.19 and for all links is 5550.92, both far greater than 1, indicating that the predictors $a$ are significantly related to joint torques. The adjusted $R^2$ values are 0.81 for link 7 and 0.85 for all links, suggesting a strong goodness-of-fit and confirming that the linear regression model effectively captures the torque dynamics.

### D. Confidence Intervals




(a) Confidence Intervals for Link 7

(b) Confidence Intervals for All Links

Fig. 4. Confidence Intervals vs. True Value

The confidence intervals (shown in Fig. 4) indicate the uncertainty of each estimate. For link 7, all estimated $a$ parameters are in confidence intervals. However, the empirical coverage rate in the regression for all links is 51% which is lower than the nominal 95%, suggesting that the model may underestimate uncertainty due to biased parameter estimates, insufficient excitation, or numerical instability in the regression matrix.

## III. Conclusions

In this lab, the dynamic parameters of the Franka Emika Panda robot were estimated via linear regression using simulated motion and torque data. When estimation was restricted to the end-effector link (Part 1), the regression achieved a high goodness-of-fit, with an adjusted $R^2$ of 0.81 and statistically significant F-values. Most parameters exhibited narrow confidence intervals and consistent signs, indicating that the method can successfully recover physical parameters.

However, extending the estimation to all seven links simultaneously (Part 2) revealed significant numerical instability. The stacked regressor matrix lost rank (43/70) and exhibited an effectively infinite condition number, reflecting severe multicollinearity and parameter coupling. Consequently, the estimated parameters displayed large uncertainties, and some confidence intervals became degenerate or undefined. This illustrates a fundamental limitation of linear regression when applied to high-dimensional robot dynamics.

The loss of rank in $Y(q, \dot{q}, \ddot{q})$ arises primarily from insufficient excitation and motion correlations. When the reference trajectory does not explore all combinations of joint accelerations and velocities, several columns of $Y$ become linearly dependent. Additionally, the nonlinear dependence of $Y$ on configuration induces coupling between parameters such as inertia and center of mass, further reducing effective dimensionality. These dependencies are magnified by sensor noise and finite numerical precision, which amplify small perturbations along poorly excited directions. As a result, the identification problem becomes ill-conditioned, and small errors in the data produce large variations in the estimated parameters.

Interestingly, despite these large deviations from the true physical parameters, the predicted joint torques remain highly accurate. This apparent contradiction arises because the mapping from parameters $a$ to torques $u = Ya$ is not unique when $Y$ is rank-deficient or nearly so. In such cases, multiple distinct parameter vectors can yield nearly identical torque outputs - i.e., the system of equations admits infinitely many solutions. The least-squares estimate obtained via the pseudoinverse simply selects the minimum-norm solution among them, which fits the observed torques well but does not necessarily correspond to the true physical parameters.

In summary, while the least-squares regression minimizes torque residuals effectively, it cannot guarantee physically meaningful parameter estimates in the presence of rank deficiency. The matrix loses rank due to correlated joint motions, insufficient trajectory diversity, and nonlinear coupling between dynamic terms, leading to loss of identifiability. Reliable full-robot identification therefore requires persistently exciting trajectories, base-parameter reduction, or the use of regularization techniques to stabilize the inversion process. Approaches such as Tikhonov regularization (ridge regression), truncated singular value decomposition (TSVD), or Bayesian estimation introduce controlled bias that suppresses the amplification of noise along poorly excited directions, yielding more stable and physically plausible parameter estimates. These findings emphasize the trade-off between model completeness, numerical robustness, and physical interpretability in practical robot system identification.

# Lab 4 – Digit recognition on the MNIST dataset

## I. Methodology

This study focuses on the task of handwritten digit recognition using the MNIST dataset. For the purpose of model depth analyses, two different architectures of the MLP were presented: one shallow MLP and one deep MLP with multiple hidden layers. The models were trained in the same manner with stochastic gradient descent, while further analyses were conducted concerning the effect of varying the sizes of the batches and the values of the learning rate on the accuracy of models performance.

### A. Data Preparation

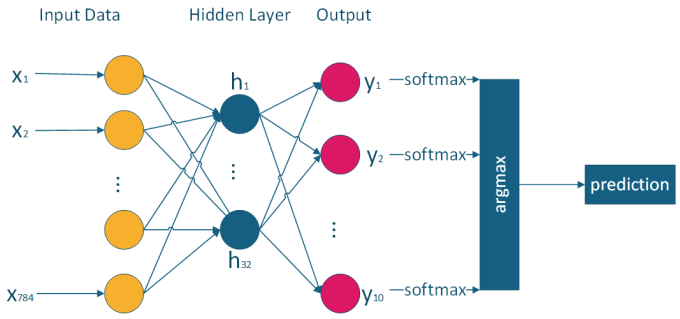
The MNIST dataset consists of 70,000 grayscale images of handwritten digits (0–9) with a resolution of $28 \times 28$ pixels. Each image is labeled with the corresponding digit, forming a standard benchmark for evaluating image classification models.

This dataset was downloaded using the `torchvision` library. Each image was transformed into a tensor and normalized with a mean of 0.1307 and standard deviation of 0.3081. The training data was split into training (80%) and validation (20%) subsets for the hyperparameter tuning. Data were loaded in mini-batches using the `DataLoader` class from `torch.utils.data`.

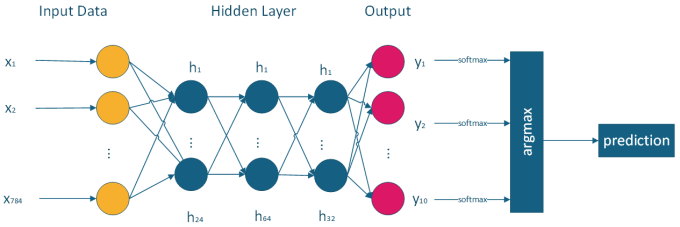
While tuning the hyperparameters, there was a separate validation subset defined with the help of the `get_train_valid_loader()` function in the `ultimate-utils` package section[1]. The `get_train_valid_loader()` function automatically divided the original training data with the same shuffle parameters and normalization settings as the initial `DataLoader` instance. This ensured that both training and validation batches were generated consistently across experiments.

### B. Network Architectures

The shallow MLP consists of an input layer of size 784 (flattened image), a hidden layer with 32 units, and an output layer with 10 neurons (digits 0–9). So, the number of parameters in the shallow network is 25,450 ($32 \times 784 + 32 + 10 \times 32 + 10 = 25,450$). The deep MLP includes three hidden layers with 24, 64, and 32 units respectively with the number of parameters 22850 ($24 \times 784 + 24 + 64 \times 24 + 64 + 32 \times 64 + 32 + 10 \times 32 + 10$) which is slightly lower than the number of parameters in the shallow network. All hidden layers employ ReLU activations, and both MLP models are trained using the negative log-likelihood loss function. The output layer applies a Log-Softmax transformation to produce a probability distribution. Architectures of the shallow and deep MLPs are illustrated in Figure 1.




(a) Shallow multiplayer perceptron




(b) Deep multiplayer perceptron

Fig. 1: Model structures.

### C. Training Procedure

All models were trained using the negative log-likelihood loss (`NLLLoss`) and the stochastic gradient descent optimizer (`SGD`) for 20 epochs initially. And for hyperparameters optimization, we tried learning rates with 0.005, 0.1, 0.2 and batch sizes with 16, 32, 64 respectively to choose the best model.

Each training epoch in the model involved resetting the gradients with `optimizer.zero_grad()`, calculating the forward pass to make predictions, calculating the loss with `criterion(output, target)`, and back propagating the gradients with `loss.backward()` and stepping the optimizer with `optimizer.step()`. The code was designed to print out values for the loss and accuracy after each epoch to see if the network was converging, and it ensured that it picked the best model on the validation data with the lowest values for validation loss after every step.

### D. Evaluation and Error Analysis

The evaluation routine was performed in `torch.no_grad()` context to avoid calculating gradients during evaluation. The predictions and corresponding labels were evaluated to calculate accuracy on the final test data, and the confusion matrix was prepared with the help of the `sklearn.metrics` library. The misclassifications were determined, and the corresponding images were viewed with the help of the custom-made function `show_prediction()`, which showed the images with the correct and predicted labels to easily validate the model performance on ambiguous writing styles.

## II. Results

### A. Task 1: Shallow vs. Deep Network

Both networks were trained on the same dataset. Figure 2 shows that although the deep MLP has higher loss in the beginning, it converges faster and achieves a lower final loss than the shallow one.

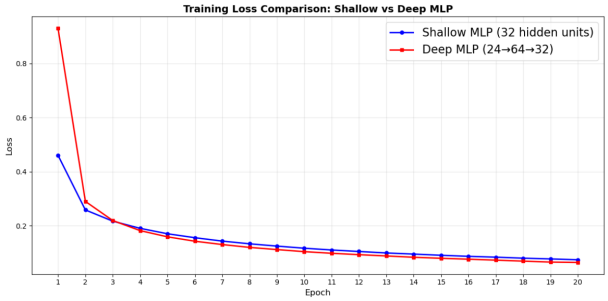



Fig. 2: Training loss comparison between shallow and deep MLPs.

### B. Task 2: hyperparameters Tuning

In this part, we choose shallow MLP model to optimize hyperparameters.

As shown in Figure 3, the training loss of all models consistently decreases over epochs, whereas the validation loss may begin to rise, reflecting reduced generalization performance. So, in order to avoid overfitting, we use validation set to choose the best model. The lowest validation loss 0.11 is generated by $batch_size = 16, learning_rate = 0.01, epoch = 19$, so we choose these hyperparameters to generate our best model.

The final test accuracy of the best model reached 96.67%. A confusion matrix (Figure 4) reveals that most errors occurred between digits with similar shapes, particularly between 4/9 and 3/5.

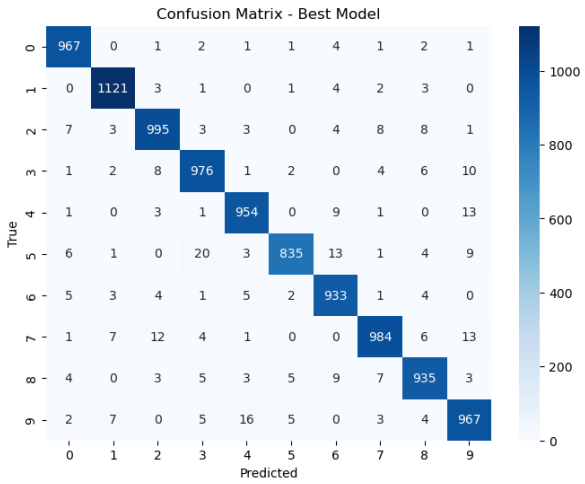
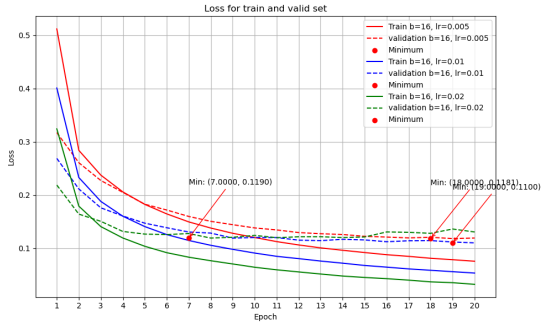
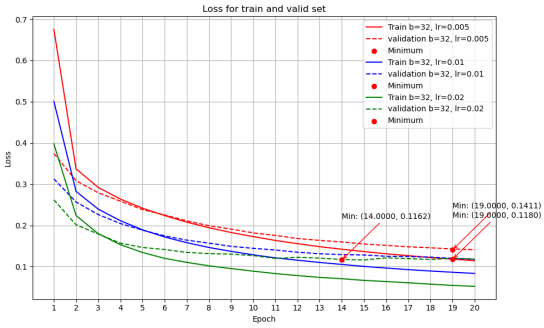


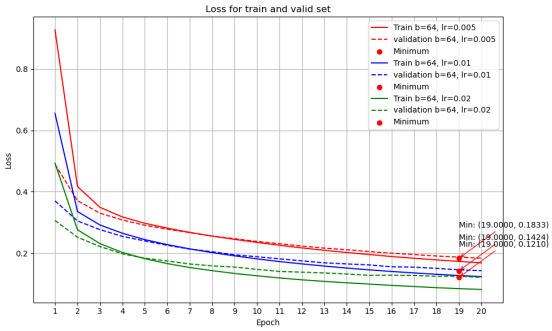

Fig. 4: Confusion matrix for the best-performing deep MLP.




(a) $batchsize = 16$




(b) $batchsize = 32$




(c) $batchsize = 64$

Fig. 3: Loss curves for training and validation data for all models.

To further analyze the model's weaknesses, two misclassified examples are illustrated in Figure 5. In the first case, the model predicted the digit **6** when the correct label was **5**. The rounded curvature at the bottom of the handwritten "5" likely resembled a "6" to the network. In the second case, the model predicted a **9** instead of a **4**, possibly due to the closed loop and top-heavy writing style of the "4" sample, which made it visually similar to a "9". These cases exemplify how subtle variations in handwriting can lead to confusion, especially for fully connected architectures that lack explicit spatial feature extraction.

Misclassified examples such as these suggest that further performance gains could be achieved with convolutional architectures that better capture local spatial correlations and stroke-level patterns. Incorporating data augmentation or attention-based mechanisms could also improve robustness against such subtle stylistic variations.

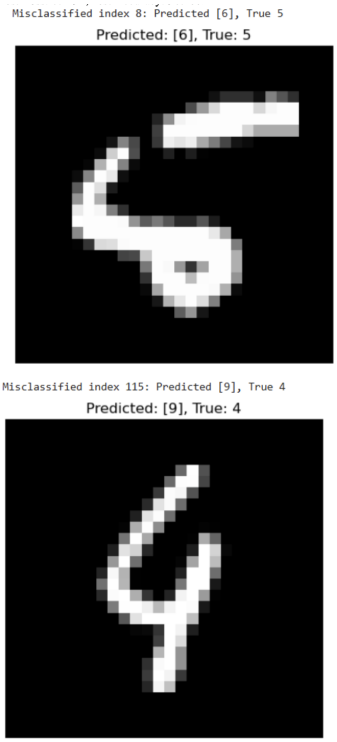



Fig. 5: Examples of misclassified digits: (a) true label 5 predicted as 6, (b) true label 4 predicted as 9.

## III. Conclusion

Despite both models having a comparable number of parameters, the deep MLP exhibited greater *representational efficiency*: by stacking multiple nonlinear transformations, it could hierarchically compose features, enabling more compact and abstract internal representations of handwritten digits. In contrast, a shallow network must learn these complex relationships directly within a single layer, leading to less effective parameter utilization.

The use of a separate validation dataset proved essential for systematic hyperparameter tuning. By monitoring validation loss, it is possible to identify overfitting and select the optimal combination of depth, hidden units, and regularization strength. This validation-guided approach ensures that model improvements reflect genuine generalization ability rather than adaptation to noise in the training data.

Training dynamics were also influenced by the choice of *batch size* and *learning rate*. We observed that smaller batch sizes (e.g. 16) resulted in faster but noisier convergence, with training loss fluctuating more across epochs. In contrast, larger batch sizes (e.g. 64) produced smoother loss curves but slower convergence. Moreover, learning rate controlled the stability and speed of optimization: too large a value led to divergence, whereas too small a value caused stagnation. Careful tuning of these parameters was therefore critical for achieving both fast and stable convergence.

While MLP models achieved strong performance on the MNIST dataset, its fully connected structure lacks spatial awareness, limiting its ability to capture local patterns in the image plane. A natural next step is to explore convolutional neural networks (CNNs), which incorporate weight sharing and spatial locality to learn translation-invariant features. Such architectures are not only more parameter-efficient for image data but also exhibit superior generalization to unseen writing styles. Further extensions could include normalization techniques, dropout for regularization, or even modern attention-based models that dynamically weigh feature importance.

In summary, this study highlights the importance of architectural depth, hyperparameter tuning, and training configuration in neural network design. Future work can extend these findings by applying convolutional or hybrid architectures, exploring data augmentation strategies, and leveraging automated hyperparameter optimization to push model performance beyond the limits of fully connected networks.

## References

[1] Brando Zhang, *ultimate-utils: Utility functions for PyTorch*, GitHub repository, https://github.com/brando90/ultimate-utils/blob/ef2217c07b43aa5354f7b6f8f1761c5f16017874/ultimate-utils-proj-src/uutils/torch_uu/dataloaders/mnist.py, Accessed: Oct. 28, 2025.

# Project Assignment – Learning a Reach Task

## I. Methodology

The objective of this project is to develop and evaluate machine learning models capable of performing a robotic reach task, where the robot's end-effector moves from an initial position to a specified three-dimensional target position and orientation. The project replaces conventional model-based control methods-such as inverse kinematics and feedback linearization-with data-driven approaches derived from simulations in the `roboenv2` environment. The work is divided into three main parts: (1) training a Multilayer Perceptron (MLP) regressor to predict joint torque commands from joint angle errors, (2) training and comparing MLP and Random Forest regressors to predict the next target joint configuration from the current joint states and desired Cartesian goals, and (3) integrating the ML model into the control loop to evaluate its qualitative performance in simulation through trajectory visualization and analysis.

### A. Data Processing

All data used for model training and evaluation were generated through a custom pipeline designed to reproduce diverse robot motions. The target positions were sampled within a 180° semicircular workspace centered at the robot base, with a radius of approximately 0.8 m. These limits were estimated based on the robot's reach and joint constraints as published by RoboDK [1]. The framework produces randomized Cartesian targets to improve model generalization and learning robustness. A total of 200 trajectories were generated using the simulation interface.

*1) Trajectory Generation:* In part 1 we used constant orientations ([0, 0, 0, 1]) for simplicity and assigned a duration of 5 seconds to each target resulting in an even distribution of data points across all target positions.

For part 2 each trajectory corresponds to a sequence of joint motions computed from a desired Cartesian end-effector target.

For each trajectory:

- The desired Cartesian position and orientation were randomized.
- The movement duration was uniformly sampled between **3 s** and **7 s**.
- The robot was reinitialized to its starting configuration before each rollout.

All joint positions, velocities, torques, Cartesian poses, and orientations were recorded at every simulation time step (typically 1 ms). To reduce storage and computational cost, data were downsampled by a factor of 2, yielding an effective time step of 2 ms per sample.

Each completed trajectory was serialized into a separate file (`data_0.pkl`, `data_1.pkl`, ...) containing the variables summarized in Table I.

The generated trajectories were automatically divided into training and testing folders according to a 70/30 split.

TABLE I
Summary of Recorded Variables in Each Trajectory File

| Variable | Description |
|---|---|
| `q_mes_all` | Measured joint positions |
| `qd_mes_all` | Measured joint velocities |
| `q_d_all` | Desired joint positions |
| `qd_d_all` | Desired joint velocities |
| `tau_mes_all` | Measured joint torques |
| `cart_pos_all`, `cart_ori_all` | Cartesian end-effector pose |
| `desired_cartesian_pos`, `desired_cartesian_ori` | Final target pose |

*2) Feature Construction:* Two supervised learning tasks were derived from the dataset:

1) Torque Prediction (Feedback Linearization Approximation)
   The model learns to approximate the low-level controller by mapping joint angle errors to torque commands:
   $$\boldsymbol{\tau}_{\text{cmd}} = f_\theta(\boldsymbol{q}_{\text{des}} - \boldsymbol{q}_{\text{mes}}). \tag{1}$$
   Here, the input has 7 dimensions (7 joint angle errors), and the output is a 7 dimensional torque vector.
2) Next Desired Joint States Regression
   This task approximates the behavior of the `CartesianDiffKin` function by predicting the next desired joint positions and velocities given the current joint configuration and target position:
   $$[\boldsymbol{q}_{\text{des}},\ \dot{\boldsymbol{q}}_{\text{des}}] = g_\phi([\boldsymbol{q}_{\text{mes}},\ \boldsymbol{x}_{\text{target}}]). \tag{2}$$

*3) Normalization and Dataset Splitting:* In part 1, before training, we applied feature standardization to the input using `StandardScaler` from `sklearn` which can transform each feature to have zero mean and unit variance, preventing the model from being biased toward features with larger numerical scales during training. To keep the physical meaning of torque commands, we did not do any process to output $Y$.

Then, we directly used `train_test_split` from `sklearn` to divide the dataset into 80% training and 20% test sets. The training set was further split into 80% training and 20% validation subsets. Both data splitting operations maintained random shuffling with a fixed seed (`random_state = 42`) to ensure reproducibility.

While for part 2, all input and output features were normalized using the mean and standard deviation computed from the training data only. Normalization was applied to all variables except torques, which were preserved in their original Newton-metre scale to maintain physical interpretability.

The datasets were further divided into training (70%), validation (15%), and test (15%) subsets using the `train_test_split` function from the `scikit-learn` library, ensuring reproducibility with a fixed random seed (`random_state=42`).

### B. MLP Regressor for Torque Commands prediction

During the experiment, we chose Adam optimizer, ReLU activation function and `MSELoss` as loss function. The MLP

Regressor consists 3 hidden layers with 128, 128, 64 sequentially shown as Figure 1.

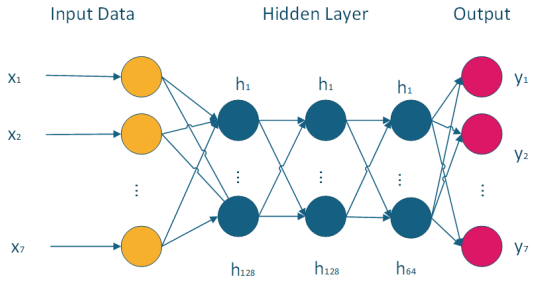



Fig. 1. Structure of MLP Regressor for Torque Commands

We initialized training $epochs = 500$, and explored multiple hyperparameter configurations, including batch sizes in $\{16, 32, 64\}$ and learning rates in $\{0.001, 0.002\}$. The best validation performance was achieved with a batch size of 32 and a learning rate of 0.001, after training for 488 epochs.

### C. MLP Regressor for Next Desired Joint Positions and Velocities

In this task, we designed a deep multilayer perceptron (MLP) consisting of three hidden layers with 64, 128, and 64 units, respectively. And we chose ReLU as activation function and MSELoss as loss function. So, the deep MLP can be shown as Figure 2

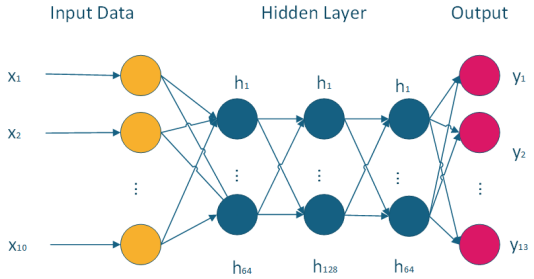



Fig. 2. MLP Regressor for Next Desired Joint Positions and Velocities

We initialized training using the `Adam` optimizer and explored the following hyperparameter search space to optimize our model: hidden layer sizes $[64, 64]$, $[128, 128]$, $[64, 128, 64]$, and $[128, 256, 128]$; batch sizes $\{64, 128, 256\}$; learning rates in the range $[0.0001, 0.01]$; and number of epochs $\{50, 100, 150\}$.

### D. Random Forest Regressor for Next Desired Joint Positions and Velocities

To construct our random forest regressor, we employed a grid search to tune the number of trees, maximum tree depth, and maximum number of features. The grid search used 5-fold cross-validation and the scoring metric was set to $neg_mean_squared_error$, as `scikit-learn` treats higher scores as better.

### E. Replacing the Classic Controller by ML Models

After training our models, we use `pickle` to save and load our models to generate desired current joint positions $q_d_all$ and velocities $qd_d_all$ instead of the original classic controller function `CartesianDiffKin`. Finally, we interpret the first seven elements (indices 0–6) of the model output as the next desired joint positions $q_des$, and the remaining seven elements (indices 7–13) as the next desired joint velocities $qd_des_clip$, which were then used to control the robotic motion.

## II. Results

### A. Evaluation of MLP Regressor for Joint Torque Commands

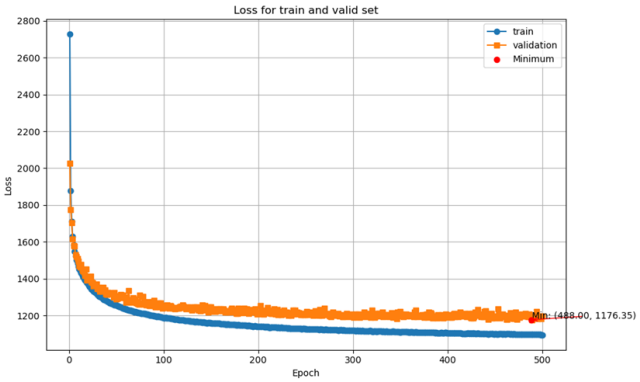



Fig. 3. Training Performance

Figure 3 the loss plot for training and validation. Then we evaluated this model on test dataset getting a test loss of 1166.1206. Meanwhile, we calculated mean absolute error and $R^2$ scores on each elements in $Y$ showing in Table II.

| Output Dimension | Mean Absolute Error | $R^2$ Score |
|---|---|---|
| $y_0$ | 12.1579 | 0.7133 |
| $y_1$ | 22.2423 | 0.9335 |
| $y_2$ | 9.461305 | 0.9181 |
| $y_3$ | 11.6954 | 0.8989 |
| $y_4$ | 5.5126 | 0.5820 |
| $y_5$ | 5.2370 | 0.5026 |
| $y_6$ | 3.2067 | 0.6789 |
| **Average** | **9.9305** | **0.7467** |

TABLE II
PER-OUTPUT MEAN ERROR AND $R^2$ SCORES ON THE TEST SET.

These results show that our regressor can approximate torque commands to some extent but not very explicitly. However, after investigation, we found that if we add joint velocity errors as input along with joint angle errors, the model would be better with higher $R^2$ score up to 0.8 or even 0.9.

Finally, we compare our predicted torque commands with the groundtruth shown in Figure 4. The plots indicate that most predicted values closely align with the groundtruth, except for a few extreme samples, which is consistent with our former evaluation.

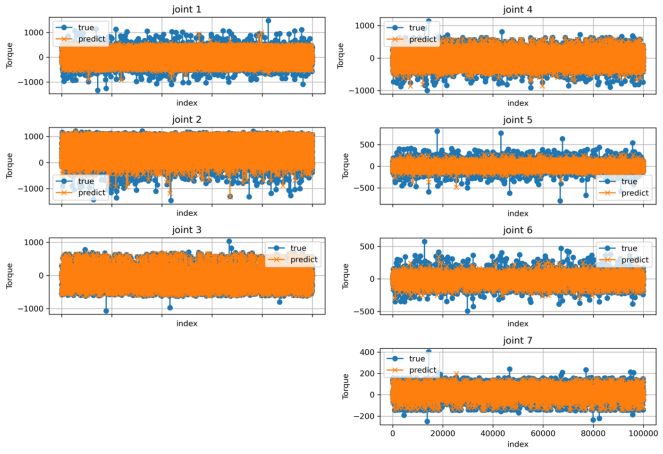



Fig. 4. Comparison of torque commands

| Output Dimension | Mean Absolute Error | $R^2$ Score |
|---|---|---|
| $y_0$ | 0.0006 | 0.9994 |
| $y_1$ | 0.0004 | 0.9996 |
| $y_2$ | 0.0004 | 0.9996 |
| $y_3$ | 0.0004 | 0.9996 |
| $y_4$ | 0.0005 | 0.9995 |
| $y_5$ | 0.0004 | 0.9996 |
| $y_6$ | 0.0005 | 0.9995 |
| $y_7$ | 0.0145 | 0.9858 |
| $y_8$ | 0.0085 | 0.9915 |
| $y_9$ | 0.0047 | 0.9954 |
| $y_{10}$ | 0.0049 | 0.9951 |
| $y_{11}$ | 0.0054 | 0.9948 |
| $y_{12}$ | 0.0066 | 0.9935 |
| $y_{13}$ | 0.0000 | 0 |
| **Average** | **0.0034** | **0.9252** |

TABLE III
PER-OUTPUT MEAN ERROR AND $R^2$ SCORES FOR THE DEEP MLP MODEL ON THE TEST SET.

### *B. Evaluation of MLP Regressor for Next Desired Joint Positions and Velocities*

*1) Hyperparameters Tuning:* By changing batch size and learning rate, we got Figure 5 and chose $epochs = 150$, $batch\ size = 256$, $learning\ rate = 0.0018$, and hidden layer sizes $[64, 128, 64]$ to form our final model.

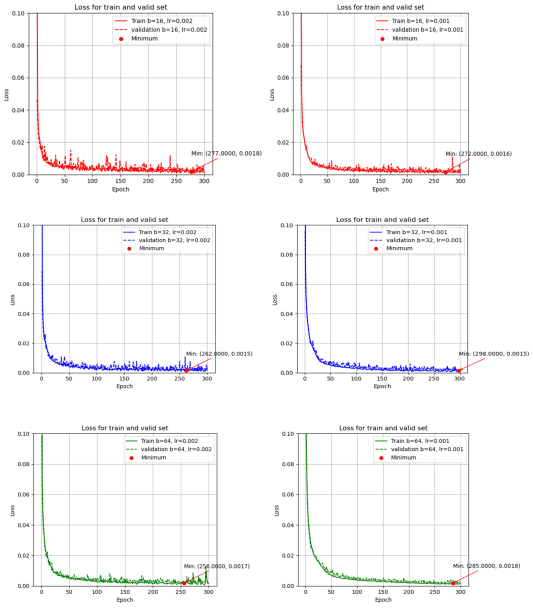



Fig. 5. Hyperparameters Comparasion

*2) Model Evaluation:* Finally, we evaluated the model on the test set and obtained an average batch-wise test loss of 0.0024. We also computed the mean absolute error and $R^2$ score for each element in $Y$, as shown in Table III.

The overall performance across joints shows strong predictive accuracy, with position components achieving $R^2$ values above 0.99 and velocity components consistently exceeding 0.98. However, we observed that the last output dimension ($y_{13}$) yields an $R^2$ score of zero.

Upon inspection, this output corresponds to the velocity of the seventh joint in the Franka Emika Panda robot, which controls the rotational roll of the wrist flange. Although the training data included variations in end-effector *orientation*, the Panda's 7-DoF redundancy allows the inverse kinematics solver to achieve these orientation changes primarily using other joints. As a result, the seventh joint itself remains largely stationary across all trajectories, minimizing unnecessary motion. Consequently, the target variance for this joint is effectively zero, and the $R^2$ score is undefined, reported as zero by `scikit-learn`.

This behavior is physically consistent: in redundant manipulators like the Panda, some joints may remain inactive for specific Cartesian position and orientation tasks due to motion minimization. Incorporating alternative kinematic strategies or task specifications could activate this joint and yield more representative metrics.

Excluding this static joint, the model achieves an average $R^2$ score of 0.994 across all other outputs, demonstrating excellent generalization and precise prediction of both joint positions and velocities.

### *C. Evaluation of Random Forest Regressor for Next Desired Joint Positions and Velocities*

*1) Parameters Searching:* To find the best parameters for the random forest regressor, we applied `GridSearchCV` from `scikit-learn` to evaluate combinations of

- the number of trees: `n_estimators` {50, 100, 200},
- maximum tree depth: `max_depth` {None, 10, 20},
- the maximum number of features: `max_features` {'auto', 'sqrt'}.

The grid search identified the optimal parameters as `max_depth` = 20, `max_features` = 'sqrt', and `n_estimators` = 50, yielding a best score of 0.00076.

*2) Model Evaluation:* Using the selected parameters, we trained the random forest regressor on the training dataset and evaluated its performance on the test set. The model achieved a mean squared error (MSE) of 0.0014 and an overall $R^2$ score

of 0.9976. Also, we calculated $R^2$ scores on each elements in $Y$ showing in Table IV.

| Output Dimension | $R^2$ Score |
|---|---|
| $y_0$ | 0.9999 |
| $y_1$ | 0.9999 |
| $y_2$ | 0.9999 |
| $y_3$ | 0.9999 |
| $y_4$ | 0.9999 |
| $y_5$ | 0.9999 |
| $y_6$ | 0.9999 |
| $y_7$ | 0.9887 |
| $y_8$ | 0.9976 |
| $y_9$ | 0.9947 |
| $y_{10}$ | 0.9969 |
| $y_{11}$ | 0.9911 |
| $y_{12}$ | 0.9976 |
| $y_{13}$ | 1 |

TABLE IV
PER-OUTPUT $R^2$ SCORES FOR THE RANDOM FOREST MODEL ON THE TEST SET.

The random forest regressor produced an $R^2$ score of 1 for output dimension $y_{13}$ due to the following reasons:

- The training data revealed that $Y[:, 13]$ is a constant vector, with all target values equal to zero.
- The model learned to output zero directly, effectively replicating the training target.
- As a result, the predictions satisfy $\hat{y}_i = 0$, leading to zero residuals. Since the total variance of the target is also zero, the coefficient of determination becomes:

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2} = 1 - \frac{0}{0}$$

Although this expression is mathematically undefined, `scikit-learn` interprets it as a perfect fit and returns $R^2 = 1$.

### D. The Performance of ML models on Robotic Motion Control

These results for the two regressors indicate excellent predictive accuracy and strong generalization capability across the test samples. We used them to replace `CartesianDiffKin` successfully and gained plots of robot trajectories. Figure 6 shows two examples of these plots.

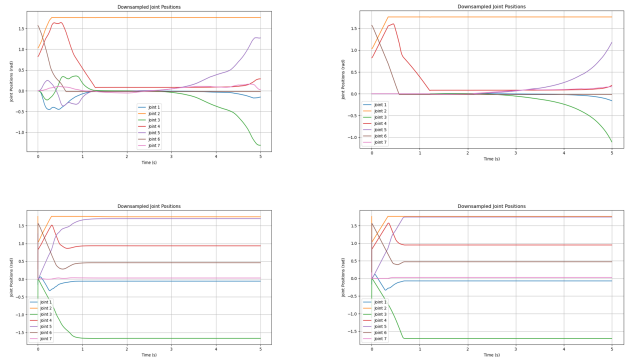


Fig. 6. Robot Trajectories

The left side of Figure 6 were robot trajectories generated by our MLP regressor with target positions [0.5,0.0,0.1] and [0.4,0.2,0.1], while the right side of Figure 6 were robot trajectories generated by classic controller `CartesianDiffKin` with the same target positions. The final joint states in both cases are closely aligned, indicating that the MLP-based controller can successfully drive the robot to the desired target positions, like the classic controller.

However, given by `panda.urdf`, we can get the upper and lower limits of each joint(Table V). Taking target position [0.5,0.0,0.1] for instance, both MLP and classic controllers make joint 2 reach the upper angle limit and joint 6 reach the lower angle limit, which may bring potential risks to the robot like singularity or mechanical wear.

| Joint Index | Lower Limit (rad) | Upper Limit (rad) |
|---|---|---|
| 1 | $-2.8973$ | 2.8973 |
| 2 | $-1.7628$ | 1.7628 |
| 3 | $-2.8973$ | 2.8973 |
| 4 | 0.0000 | 3.0020 |
| 5 | $-2.8973$ | 2.8973 |
| 6 | $-0.0175$ | 3.7525 |
| 7 | $-2.8973$ | 2.8973 |

TABLE V
JOINT ANGLE LIMITS

Another interesting phenomenon we found during the task is that the random forest controller execute significantly slower prediction than MLP controller and classic controller. We researched the reason that due to its ensemble structure, each prediction requires traversing multiple decision trees where each tree performs a sequence of conditional checks from root to leaf. This process is inherently sequential and difficult to parallelize efficiently. In contrast, MLPs perform a single forward pass through a fixed network of matrix operations, which are highly optimized and GPU-friendly. Classical controllers, such as `CartesianDiffKin`, rely on analytical formulas or closed-form solutions, making them extremely fast and lightweight. As a result, while Random Forests may offer strong predictive accuracy in some cases, their computational overhead makes them less suitable for real-time robotic control tasks where speed is critical.

To further evaluate our controller's generalization capability, we tested it on target positions with a non-constant $z$-component. Unlike the previous version, which was only trained on trajectories with $z = 0.1$, our updated MLP controller successfully handles motions in three dimensions. As shown in Figure 7, the MLP-generated trajectory closely follows the reference trajectory obtained from the classical controller, demonstrating stable behavior across all spatial axes.

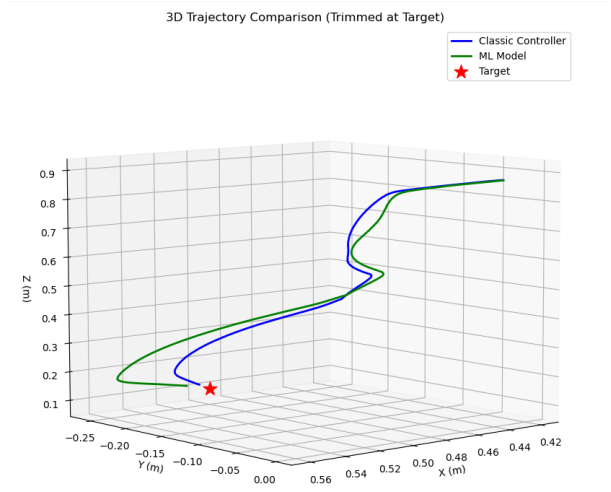



Fig. 7. Comparison of classical and MLP-generated trajectories in 3D space.

This experiment confirms that, with the inclusion of training data covering multiple $z$-values, the MLP controller can generalize to fully three-dimensional motion. While minor end-point fluctuations may still occur, they are significantly reduced compared to earlier iterations, indicating improved stability and spatial understanding.

## III. Conclusion

The project has shown how data-based learning algorithms can positively mimic and even outperform traditional computational methods for controlling robotic movements. The goal of this project has been to determine whether application of machine learning techniques such as neural networks and ensemble approaches can estimate control commands and produce smooth motions for a robot to reach a target position. The project was developed in three stages to first predict torque commands, then train a trajectory for joints, and eventually replace a traditional controller with a learned one.

In Part 1, a multilayer perceptron (MLP) regressor is designed to calculate joint torque values as a function of angle errors between actual and required joint angles. In this approach, the MLP neural network is capable of directly identifying and relating actual angle errors to respective torque commands.

In Part 2, we trained both a Multi-Layer Perceptron (MLP) and a Random Forest (RF) model to estimate the next desired joint positions and velocities based on the robot's current joint configuration and the final Cartesian target. This approach allows for a direct comparison of the two models' performance. The MLP serves as a learned inverse kinematics module, replacing traditional analytical computations with a purely data-driven mapping, while the RF model provides an alternative baseline predictor trained on the same dataset.

In Part 3, the MLP network designed in Part 2 replaced the conventional `CartesianDiffKin` function that calculated the required joint states in the control loop of the robot. This modification allowed the neural network to compute the desired joint positions and velocities in real-time, which were then used by the feedback linearization torque controller. The robot moved towards its targeted locations as it followed smooth and properly defined trajectories. The robot's motions were stable and dynamically consistent as qualitatively observed, to ensure that the MLP network performs well in a closed-loop control system. Interestingly, although the random forest regressor has a lower MSE and a higher $R^2$ value than the MLP regressor, it is unable to provide a fast prediction that can smoothly guide movements in a robotic setup, so it is not as suitable a choice for this real-time application.

While results are highly promising, several limitations were observed. The models were trained on a limited dataset, which may constrain their ability to generalize to unseen conditions or higher-speed motions. The MLP's feedforward nature meant that temporal dependencies and dynamic effects such as inertia or frictional lag were not explicitly modeled. Furthermore, certain joints, particularly joint 7, exhibited little variation in the training data, reducing the model's ability to capture accurate velocity dynamics for those degrees of freedom. Even though the simulation included noise, the condition were still highly idealized, as it doesn't include frictional uncertainties, or actuation delays, and the evaluation of the learned controller was qualitative rather than quantitative.

To further improve performance, future works should focus on expanding and diversifying the training dataset to include a wider range of trajectories, dynamic conditions, and disturbances. Introducing more complex noise profiles and perturbations during training could enhance robustness and facilitate real-world transferability. Incorporating recurrent or sequence-based architectures such as LSTMs or GRUs would allow the model to capture temporal dependencies, improving performance during dynamic tasks. A hybrid approach that combines classical control laws with learned corrections could also merge the stability and interpretability of analytical methods with the adaptability of neural networks. Quantitative trajectory analysis measuring tracking errors, energy efficiency, and smoothness should be included to rigorously compare the learned and classical controllers.

Overall, this study has proven that machine learning algorithms and specifically the MLP and Random Forest methods can forecast and control a robot's movements with outstanding accuracy. Although the Random Forest better approximated the numerical solution to this end, its discontinuous behavior made it less applicable for a real-time task. The proposed integration of a learned MLP control algorithm within a simulation environment confirmed that ML techniques could not only approximate but even substitute traditional control system components in some scenarios. Thus, this investigation is a significant step toward more intelligent and adaptive robot controls that combine physical and model-based approaches with learned patterns for more efficient robot movements.

## References

[1] RoboDK, *Franka Emika Panda Robot Specifications*, https://robodk.com/robot/Franka/Emika-Panda, Accessed: Oct. 28, 2025.